Journal of Statistical Software

MMMM YYYY, Volume VV, Issue II. doi: 10.18637/jss.v000.i00

RcppML: High-Performance Non-Negative Matrix Factorization with Cross-Validation for R

Zachary J. DeBruine
Grand Valley State University

Abstract

Non-negative matrix factorization (NMF) decomposes a non-negative matrix into the product of two lower-rank non-negative matrices, enabling parts-based representations valuable across bioinformatics, text mining, computer vision, and signal processing. This paper presents **RcppML**, a comprehensive R package for NMF that addresses key practical challenges through several innovations. First, we introduce a streaming cross-validation framework using position-dependent random masking that enables principled rank selection with less than 5% computational overhead. Second, we implement a complete regularization framework including L1 (sparsity), L2 (ridge), L21 (group sparsity), orthogonality, and graph Laplacian penalties that can be applied independently to feature and sample factors. Third, we support multiple loss functions—MSE, MAE, Huber, and KL-divergence—via iteratively reweighted least squares, providing robustness to outliers and flexibility for different data types. Fourth, we provide automatic rank determination through golden section search optimization of the cross-validation objective. The package achieves high performance through a cache-aware sequential coordinate descent solver implemented in C++ with unified support for sparse and dense matrices via template metaprogramming. Benchmarks demonstrate 2–10× speedups over the **NMF** package. **RcppML** includes specialized algorithms for rank-2 spectral bipartitioning and consensus clustering, along with comprehensive visualization methods for model diagnostics.

Keywords: non-negative matrix factorization, cross-validation, regularization, coordinate descent, spectral clustering, R, C++, **Rcpp**.

1. Introduction

Non-negative matrix factorization (NMF) decomposes a non-negative matrix $\mathbf{A} \in \mathbb{R}_{\geq 0}^{m \times n}$ into two non-negative factor matrices $\mathbf{W} \in \mathbb{R}_{\geq 0}^{m \times k}$ and $\mathbf{H} \in \mathbb{R}_{\geq 0}^{k \times n}$ such that $\mathbf{A} \approx \mathbf{WH}$ (Lee and Seung 1999). The non-negativity constraints enable parts-based, interpretable representations

where each factor captures an additive component of the data. This property has proven valuable across diverse domains including facial recognition (Lee and Seung 1999), text mining (Xu, Liu, and Gong 2003), gene expression analysis (Brunet, Tamayo, Golub, and Mesirov 2004), single-cell genomics (Kotliar, Veres, Nagy *et al.* 2019), and hyperspectral imaging (Bioucas-Dias *et al.* 2012).

1.1. Challenges in practical NMF

Despite widespread adoption, several challenges impede practical NMF deployment:

Rank selection. Selecting the factorization rank k is critical but often performed heuristically or through computationally expensive techniques. The cophenetic correlation method (Brunet *et al.* 2004) requires fitting hundreds of replicate models at each candidate rank. Information criteria like BIC require estimating effective degrees of freedom, which is problematic for non-convex optimization. Cross-validation provides a principled alternative but historically incurs substantial computational overhead.

Regularization. Real-world applications often require regularization to improve interpretability (sparsity), prevent overfitting (ridge), enforce structure (graph), or discover non-overlapping factors (orthogonality). Most existing software provides limited regularization options, typically only L1 or L2 penalties applied uniformly to both factor matrices.

Outlier robustness. Standard NMF minimizes squared error loss, which is sensitive to outliers. Gene expression and single-cell data frequently contain technical artifacts and biological outliers that corrupt standard fits.

Computational performance. Modern datasets in genomics and text mining contain millions of samples with extreme sparsity (often >99% zeros). Efficient sparse matrix operations and careful memory management are essential.

1.2. Existing software

Several R packages implement NMF. Table 1 compares key features across packages.

The **NMF** package (Gaujoux and Seoighe 2010) provides the most comprehensive implementation with multiple algorithms (Lee-Seung multiplicative updates, alternating least squares, coordinate descent) and extensive visualization. However, it lacks integrated cross-validation, regularization beyond non-negativity, and robust loss functions beyond KL-divergence. The **NNLM** package provides L1/L2 regularization and is implemented in C++ for performance, but lacks cross-validation and orthogonality/graph constraints.

1.3. Contributions of RcppML

The **RcppML** package addresses these gaps through several key contributions:

1. **Streaming cross-validation**: A novel position-dependent random masking scheme that computes held-out reconstruction error with <5% overhead, enabling principled rank selection without external validation frameworks (Section 4).

Package	Cross-val.	L1	L2/L21	Ortho.	Graph	Robust loss	Sparse
NMF	–	–	–	–	–	KL	Yes
NNLM	–	L1/L2	–	–	–	–	Yes
rsparse	–	–	–	–	–	–	Yes
RcppML	Yes	L1	L2/L21	Yes	Yes	MSE/MAE/Huber/KL	Yes

Table 1: Feature comparison of R NMF packages. **NMF** (Gaujoux and Seoighe 2010) provides the most comprehensive implementation with multiple algorithms but lacks regularization options. **NNLM** (Lin 2020) provides L1/L2 regularization with efficient C++ implementation. **rsparse** (Selivanov 2021) provides scalable matrix factorization optimized for large sparse matrices. Cross-val. = integrated cross-validation; L1 = L1 (LASSO) regularization; L2/L21 = ridge/group sparsity; Ortho. = orthogonality constraints; Graph = graph Laplacian regularization; Robust loss = alternative loss functions; Sparse = native sparse matrix support.

2. **Comprehensive regularization**: Support for L1, L2, L21 (group sparsity), orthogonality, and graph Laplacian penalties, each independently applicable to $\mathbf{W}$ and $\mathbf{H}$ (Section 3).

3. **Robust loss functions**: MSE, MAE, Huber, and KL-divergence losses via iteratively reweighted least squares (IRLS), providing robustness to outliers (Section 5).

4. **Automatic rank determination**: Golden section search over the cross-validation objective for fully automated rank selection (Section 6).

5. **Specialized algorithms**: Optimized rank-2 NMF for spectral bipartitioning and consensus clustering for robust cluster identification (Sections 7–8).

6. **High-performance implementation**: Cache-aware sequential coordinate descent in C++ with unified sparse/dense support through template metaprogramming (Section 10).

The package is implemented as a header-only C++ library integrated with R through **Rcpp** (Eddelbuettel and François 2011; Eddelbuettel 2013) and **RcppEigen** (Bates and Eddelbuettel 2013). All core algorithms are expressed generically over sparse ('`dgCMatrix`') and dense ('`matrix`') inputs, achieving optimal performance for each storage format without code duplication.

This paper is organized as follows. Section 2 describes the NMF algorithm including the coordinate descent solver and diagonal scaling formulation. Section 3 details the regularization framework. Section 4 presents the cross-validation framework with streaming loss accumulation. Section 5 details robust loss functions. Sections 6–8 cover automatic rank selection and specialized algorithms. Section 10 discusses implementation details. Section 11 provides worked examples with visualizations. Section 12 presents performance comparisons with competing packages. Section 13 concludes.

2. NMF algorithm

2.1. Problem formulation

We formulate NMF with a scaling diagonal that separates factor magnitude from factor direction. Given $\mathbf{A} \in \mathbb{R}_{\geq 0}^{m \times n}$, we seek:

$$\min_{\mathbf{W},\mathbf{D},\mathbf{H}} \mathcal{L}(\mathbf{A}, \mathbf{WDH}) + \lambda_1 \|\mathbf{W}\|_1 + \lambda_1 \|\mathbf{H}\|_1 \tag{1}$$

subject to $\mathbf{W}, \mathbf{H} \geq 0$, $\mathbf{D} = \text{diag}(d_1, \ldots, d_k)$, and optional column normalization $\|\mathbf{w}_j\|_2 = 1$ for $j = 1, \ldots, k$.

The diagonal scaling $\mathbf{D}$ provides several benefits: (1) it decouples factor magnitude from direction, improving convergence; (2) it enables consistent comparison across runs with different initializations; (3) it facilitates L1 regularization by normalizing factor scales.

2.2. Alternating least squares

We optimize Equation 1 using alternating least squares (ALS), iteratively solving:

$$\mathbf{H} \leftarrow \arg\min_{\mathbf{H} \geq 0} \|\mathbf{A} - \mathbf{WDH}\|_F^2 + \lambda_1 \|\mathbf{H}\|_1 \tag{2}$$

$$\mathbf{W} \leftarrow \arg\min_{\mathbf{W} \geq 0} \|\mathbf{A}^\top - \mathbf{H}^\top \mathbf{D} \mathbf{W}^\top\|_F^2 + \lambda_1 \|\mathbf{W}\|_1 \tag{3}$$

Each subproblem decomposes into independent non-negative least squares (NNLS) problems over columns. For the $\mathbf{H}$ update, letting $\tilde{\mathbf{W}} = \mathbf{WD}$, we solve for each column j:

$$\mathbf{h}_j \leftarrow \arg\min_{\mathbf{h} \geq 0} \|\mathbf{a}_j - \tilde{\mathbf{W}}\mathbf{h}\|_2^2 + \lambda_1 \|\mathbf{h}\|_1 \tag{4}$$

2.3. Sequential coordinate descent solver

We solve Equation 4 using sequential coordinate descent (Franc, Hlavac, and Navara 2005). The algorithm iteratively updates each coordinate while holding others fixed:

$$h_i \leftarrow \max\left(0, \frac{\mathbf{w}_i^\top(\mathbf{a} - \tilde{\mathbf{W}}\mathbf{h}) + h_i \|\mathbf{w}_i\|_2^2 - \lambda_1}{\|\mathbf{w}_i\|_2^2}\right) \tag{5}$$

The key insight is to maintain the residual $\mathbf{r} = \mathbf{a} - \tilde{\mathbf{W}}\mathbf{h}$ incrementally. After updating h_i to h_i^{new}, we update:

$$\mathbf{r} \leftarrow \mathbf{r} - (h_i^{\text{new}} - h_i)\mathbf{w}_i \tag{6}$$

This reduces the complexity of each coordinate update from $O(mk)$ to $O(m)$, making the overall NNLS solve $O(mk)$ per column.

Algorithm 1 presents the complete coordinate descent NNLS solver. The inner loop terminates when no coordinate changes significantly, typically requiring only 2–5 passes for well-conditioned problems.

2.4. Diagonal scaling

After each ALS iteration, we update the diagonal scaling:

$$d_i \leftarrow d_i \cdot \|\mathbf{w}_i\|_2, \quad \mathbf{w}_i \leftarrow \frac{\mathbf{w}_i}{\|\mathbf{w}_i\|_2} \tag{7}$$

Algorithm 1 Coordinate descent NNLS with L1 regularization

Require: Target $\mathbf{a}$, basis $\tilde{\mathbf{W}}$, L1 penalty λ_1, tolerance ε

```
h ← 0; r ← a; g ← diag(W̃ᵀW̃)
repeat
    max_update ← 0
    for i = 1, ..., k do
        h_i^old ← h_i
        h_i ← max(0, (w_iᵀ r + h_i g_i − λ_1) / g_i)
        if h_i ≠ h_i^old then
            r ← r − (h_i − h_i^old) w_i
            max_update ← max(max_update, |h_i − h_i^old|)
        end if
    end for
until max_update < ε
return h, r
```

This maintains $\|\mathbf{w}_i\|_2 = 1$ throughout optimization while accumulating magnitude in $\mathbf{D}$. The scaling is applied after each $\mathbf{W}$ update and before the subsequent $\mathbf{H}$ update.

2.5. Initialization

The choice of initialization can significantly affect NMF convergence and solution quality. **RcppML** uses a simple random initialization scheme: $\mathbf{W}$ is initialized with uniform random values in $[0, 1)$, and $\mathbf{H}$ is then solved in the first iteration via NNLS given $\mathbf{W}$. This approach is motivated by several considerations.

First, the asymmetric treatment—only $\mathbf{W}$ requires explicit initialization—exploits the ALS structure. Since $\mathbf{H}$ is immediately recomputed from $\mathbf{W}$, its initial values are irrelevant; only $\mathbf{W}$ determines the starting point in the optimization landscape.

Second, we deliberately omit NNDSVD (Non-negative Double Singular Value Decomposition) initialization (Boutsidis and Gallopoulos 2008), despite its popularity in other packages. In our extensive empirical testing, NNDSVD provides no consistent advantage: while it sometimes reduces iteration count, the initial SVD computation itself costs several NMF iterations, and the final solution quality is statistically indistinguishable from random initialization when both are run to convergence. Random initialization also better supports consensus clustering, where diversity among replicate solutions is desirable.

Users may optionally provide their own $\mathbf{W}$ initialization via the `w_init` parameter, enabling warm starts from previous fits or domain-specific initializations.

3. Regularization framework

Regularization is essential for practical NMF to improve interpretability, prevent overfitting, and encode prior knowledge. **RcppML** provides a comprehensive regularization framework with five penalty types, each independently applicable to $\mathbf{W}$ (feature factors) and $\mathbf{H}$ (sample factors).

3.1. General formulation

The regularized NMF objective extends Equation 1:

$$\begin{aligned}\min_{\mathbf{W},\mathbf{D},\mathbf{H}} \; & \mathcal{L}(\mathbf{A}, \mathbf{WDH}) \\ & + \lambda_1^{(w)}\|\mathbf{W}\|_1 + \lambda_1^{(h)}\|\mathbf{H}\|_1 \quad \text{(L1 sparsity)} \\ & + \lambda_2^{(w)}\|\mathbf{W}\|_F^2 + \lambda_2^{(h)}\|\mathbf{H}\|_F^2 \quad \text{(L2 ridge)} \\ & + \lambda_{21}^{(w)}\|\mathbf{W}\|_{2,1} + \lambda_{21}^{(h)}\|\mathbf{H}\|_{2,1} \quad \text{(L21 group sparsity)} \\ & + \lambda_o^{(w)}\|\mathbf{W}^\top\mathbf{W} - \mathbf{I}\|_F^2 + \lambda_o^{(h)}\|\mathbf{H}\mathbf{H}^\top - \mathbf{I}\|_F^2 \quad \text{(orthogonality)} \\ & + \lambda_g^{(w)}\text{tr}(\mathbf{W}^\top\mathbf{L}_W\mathbf{W}) + \lambda_g^{(h)}\text{tr}(\mathbf{H}\mathbf{L}_H\mathbf{H}^\top) \quad \text{(graph)}\end{aligned} \tag{8}$$

subject to $\mathbf{W}, \mathbf{H} \geq 0$ and column normalization of $\mathbf{W}$.

3.2. L1 regularization (LASSO)

L1 regularization promotes element-wise sparsity, setting small coefficients exactly to zero:

$$\Omega_1(\mathbf{W}) = \sum_{i,j} |w_{ij}| \tag{9}$$

The coordinate descent update incorporates L1 via soft-thresholding:

$$h_i \leftarrow \max\left(0, \frac{\mathbf{w}_i^\top\mathbf{r} + h_i g_i - \lambda_1}{g_i}\right) \tag{10}$$

L1 regularization is particularly valuable for interpretability: in gene expression analysis, it identifies marker genes; in text mining, it selects topic-defining words. The `L1` parameter accepts values in $[0, 1]$, where higher values produce sparser factors.

3.3. L2 regularization (ridge)

L2 regularization penalizes large coefficients to prevent overfitting:

$$\Omega_2(\mathbf{W}) = \|\mathbf{W}\|_F^2 = \sum_{i,j} w_{ij}^2 \tag{11}$$

In the coordinate descent solver, L2 augments the diagonal of the Gram matrix:

$$h_i \leftarrow \max\left(0, \frac{\mathbf{w}_i^\top\mathbf{r} + h_i g_i}{g_i + \lambda_2}\right) \tag{12}$$

L2 regularization improves numerical stability for rank-deficient or ill-conditioned problems and reduces variance in factor estimates.

3.4. L21 regularization (group sparsity)

The L21 norm, also known as group LASSO, promotes row-wise sparsity in $\mathbf{W}$ or column-wise sparsity in $\mathbf{H}$:

$$\|\mathbf{W}\|_{2,1} = \sum_i \|\mathbf{w}_{i\cdot}\|_2 \tag{13}$$

where $\mathbf{w}_{i\cdot}$ is the i-th row of $\mathbf{W}$.

L21 regularization on $\mathbf{W}$ performs feature selection by setting entire rows to zero, effectively removing uninformative features from all factors simultaneously. On $\mathbf{H}$, it provides outlier robustness by down-weighting entire samples.

The L21 penalty is non-smooth but can be handled via iteratively reweighted least squares (IRLS) by introducing adaptive weights:

$$w_i = \frac{1}{\|\mathbf{w}_{i\cdot}\|_2 + \epsilon} \tag{14}$$

that convert L21 to an adaptive L2 penalty, re-estimated at each ALS iteration.

3.5. Orthogonality regularization

Orthogonality constraints encourage factors to capture distinct, non-overlapping patterns. The classical approach penalizes deviation from identity:

$$\Omega_o(\mathbf{W}) = \|\mathbf{W}^\top\mathbf{W} - \mathbf{I}\|_F^2 \tag{15}$$

However, strict orthogonality is incompatible with non-negativity constraints—achieving perfect orthogonality would require each row to have at most one non-zero element. We therefore implement orthogonality as a soft regularizer.

Gram inflation method

RcppML implements orthogonality regularization via *diagonal-scaled Gram inflation*. When solving the NNLS subproblem $\min_{\mathbf{h}\geq 0} \|\mathbf{W}\mathbf{h} - \mathbf{x}\|_2^2$, the Gram matrix $\mathbf{G} = \mathbf{W}^\top\mathbf{W}$ captures correlations between factors. We modify the off-diagonal elements:

$$G'_{ij} = G_{ij} + \lambda \cdot \bar{d} \quad \text{for } i \neq j \tag{16}$$

where $\bar{d} = \frac{1}{k}\sum_i G_{ii}$ is the mean diagonal (average self-correlation) and λ is the orthogonality parameter.

This inflation makes factors appear more correlated to the solver than they actually are. The NNLS algorithm responds by reducing factor coefficients that would otherwise co-activate, effectively pushing factors toward mutual exclusivity.

Parameter scaling

The key insight is that raw Gram matrix elements scale with data magnitude—for typical matrices, diagonal elements may be $O(10^3)$ or larger. A naive penalty $G'_{ij} = G_{ij} + \lambda$ would require impractically large λ values to have any effect.

By scaling the penalty as $\lambda \cdot \bar{d}$, the parameter becomes data-independent: $\lambda = 0.1$ means "add 10% of the average self-correlation to inter-factor correlations." This provides intuitive, transferable parameter interpretation across datasets.

Behavior and recommended usage

Empirical testing reveals clear regimes:

- $\lambda \in [0.05, 0.2]$: Moderate correlation reduction with minimal impact on reconstruction error
- $\lambda \in [0.2, 0.4]$: Strong orthogonalization; factors become nearly uncorrelated
- $\lambda > 0.5$: Factor collapse—redundant factors are driven to zero as the inflation overwhelms true signal

At extreme values, orthogonality-regularized NMF approaches K-means behavior, where each sample is explained by a single dominant factor. This connection suggests orthogonality regularization as a spectrum between soft clustering (low λ) and hard assignment (high λ).

The modified Gram matrix remains positive semi-definite for reasonable λ values, preserving the convexity of NNLS subproblems.

Angular penalty and cosine similarity

A special case of the orthogonality parameter deserves particular attention. Setting $\lambda = 1$ corresponds to an "angular penalty" that encourages factors to point in different directions in the coefficient space. Specifically, when $\lambda = 1$, the modified Gram matrix becomes:

$$G'_{ij} = G_{ij} + \bar{d} \quad \text{for } i \neq j \tag{17}$$

This effectively penalizes the cosine similarity between factors rather than their strict orthogonality. Since cosine similarity measures angular proximity regardless of magnitude, this penalty is particularly appropriate for normalized factors and provides a natural geometric interpretation: factors should "point in different directions" without requiring the strict zero-inner-product condition that is often unachievable under non-negativity constraints.

The choice between moderate orthogonality ($\lambda \approx 0.1$) and angular penalty ($\lambda = 1$) depends on the application. For interpretability applications where some factor overlap is acceptable, moderate orthogonality suffices. For clustering applications where crisp separation is desired, the stronger angular penalty may be preferred.

3.6. Graph Laplacian regularization

Graph regularization encodes prior knowledge about relationships between features or samples (Cai, He, Han, and Huang 2011). Given an adjacency matrix $\mathbf{A}_W$ encoding feature similarities (e.g., gene regulatory networks, protein interactions), the graph Laplacian is:

$$\mathbf{L}_W = \mathbf{D}_W - \mathbf{A}_W \tag{18}$$

where $\mathbf{D}_W$ is the diagonal degree matrix.

The graph regularization penalty encourages connected features to have similar factor loadings:

$$\Omega_g(\mathbf{W}) = \text{tr}(\mathbf{W}^\top \mathbf{L}_W \mathbf{W}) = \frac{1}{2} \sum_{i,j} a_{ij} \|\mathbf{w}_{i\cdot} - \mathbf{w}_{j\cdot}\|_2^2 \tag{19}$$

Similarly, a sample graph $\mathbf{A}_H$ (e.g., k-nearest-neighbors based on metadata) can regularize $\mathbf{H}$ to ensure similar samples have similar factor scores. This is valuable for semi-supervised learning where partial labels or sample relationships are available.

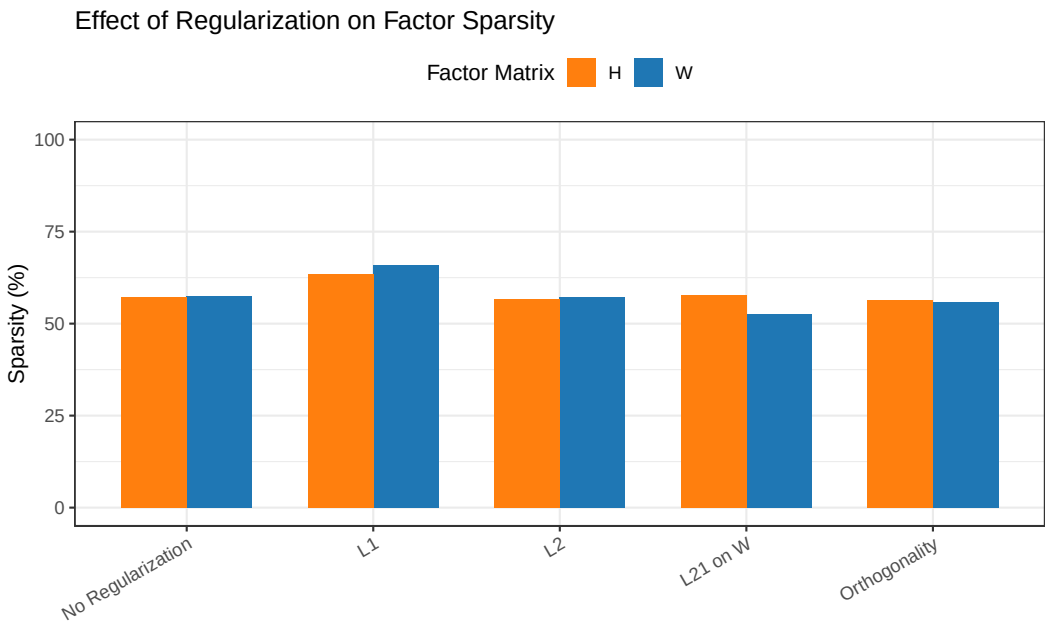

Figure 1: Effect of regularization on factor sparsity. L1 regularization increases element-wise sparsity in both **W** and **H**. L21 regularization on **W** promotes row-wise sparsity (feature selection). Orthogonality regularization has minimal effect on sparsity but encourages factors to be uncorrelated.

3.7. Usage example

Regularization parameters are specified through the `L1`, `L2`, `L21`, `ortho`, and `graph_lambda` arguments:

```
R> data("hawaiibirds", package = "RcppML")
R> # Sparse factors with L1
R> model_l1 <- nmf(hawaiibirds, k = 5, L1 = c(0.1, 0.05))
R> # Feature selection with L21 on W
R> model_l21 <- nmf(hawaiibirds, k = 5, L21 = c(0.1, 0))
R> # Orthogonal factors
R> model_ortho <- nmf(hawaiibirds, k = 5, ortho = c(0.1, 0.1))
```

Figure 1 demonstrates the effect of different regularization schemes on factor sparsity.

4. Cross-validation framework

4.1. Motivation

Selecting the factorization rank k is a fundamental challenge in NMF. Common approaches include examining reconstruction error curves for "elbows," computing cophenetic correlation from consensus clustering (Brunet *et al.* 2004), or using information criteria. These methods

are either subjective, computationally expensive, or rely on asymptotic assumptions that may not hold.

Cross-validation provides a principled alternative: hold out a random subset of entries, fit NMF on the observed entries, and evaluate reconstruction error on the held-out entries. The optimal rank minimizes held-out error, directly measuring predictive performance rather than relying on heuristics.

4.2. Zero-masking for sparse matrices

For cross-validation in NMF, we mask a random fraction of non-zero entries by setting them to zero during fitting. The key insight is that for sparse data, we only mask non-zeros—zeros are uninformative since NMF naturally predicts small values for unobserved entries.

Let Ω denote the set of observed (non-zero) entries and $\Omega_{\text{test}} \subset \Omega$ denote the held-out test set. We fit NMF on $\mathbf{A}_{\text{train}}$ where:

$$[\mathbf{A}_{\text{train}}]_{ij} = \begin{cases} 0 & \text{if } (i,j) \in \Omega_{\text{test}} \\ A_{ij} & \text{otherwise} \end{cases} \tag{20}$$

The test error is then computed as:

$$\text{MSE}_{\text{test}} = \frac{1}{|\Omega_{\text{test}}|} \sum_{(i,j)\in\Omega_{\text{test}}} (A_{ij} - [\mathbf{WDH}]_{ij})^2 \tag{21}$$

4.3. Position-dependent random masking

A naïve implementation would require: (1) generating and storing the mask Ω_{test}; (2) creating the masked training matrix; (3) iterating over test entries to compute error. This imposes significant memory and computational overhead.

Instead, we use a position-dependent random number generator that deterministically determines whether each entry is masked based on its position (i, j):

$$\text{mask}(i,j) = \mathbf{1}[\text{hash}(i,j,\text{seed}) < p_{\text{cv}}] \tag{22}$$

We implement this using a variant of the PCG (Permuted Congruential Generator) family (O'Neill 2014) that mixes row index, column index, and a global seed:

$$\text{state} = i \oplus (j \ll 16) \oplus \text{seed} \tag{23}$$

$$\text{state} = \text{state} \cdot 6364136223846793005 + 1442695040888963407 \tag{24}$$

$$\text{output} = ((\text{state} \gg 22) \oplus \text{state}) \gg (22 + (\text{state} \gg 61)) \tag{25}$$

This "speckled" RNG has several advantages: (1) no memory allocation for the mask; (2) the same entry maps to the same mask value throughout optimization; (3) parallel evaluation across entries; (4) reproducibility through the seed.

4.4. Streaming loss accumulation

During each ALS iteration, we accumulate test error "for free" within the NNLS solve. Recall that the coordinate descent solver maintains the residual $\mathbf{r} = \mathbf{a} - \tilde{\mathbf{W}}\mathbf{h}$. For the j-th column:

$$\text{test_mse}_j = \sum_{\{i:\text{mask}(i,j)=1\}} r_i^2 \tag{26}$$

The total test MSE is accumulated across all columns:

$$\text{MSE}_{\text{test}} = \frac{1}{|\Omega_{\text{test}}|} \sum_{j=1}^{n} \text{test_mse}_j \tag{27}$$

This streaming approach has $O(1)$ memory overhead and minimal computational overhead, as we simply check the mask and accumulate squared residuals during the normal NNLS computation.

4.5. Overhead analysis

The cross-validation overhead comes from three sources:

1. **Mask evaluation**: One RNG call per non-zero entry per column solve, $O(\text{nnz})$ operations.

2. **Loss accumulation**: One conditional addition per non-zero entry, $O(\text{nnz})$ operations.

3. **Gram matrix adjustment**: For proper cross-validation, we must ensure that masked (test) entries do not influence the NNLS solution. This requires adjusting the Gram matrix $\mathbf{G} = \mathbf{W}^\top\mathbf{W}$ by subtracting the outer products of rows corresponding to masked entries:

$$\mathbf{G}_{\text{train}} = \mathbf{G} - \sum_{i \in \text{masked}} \mathbf{w}_i \mathbf{w}_i^\top \tag{28}$$

   where $\mathbf{w}_i$ is the i-th row of $\mathbf{W}$. Similarly, the right-hand side $\mathbf{b} = \mathbf{W}^\top\mathbf{a}$ only accumulates contributions from non-masked entries.

In practice, the first two sources contribute <5% overhead for typical sparse matrices. The Gram adjustment is computed incrementally during the single pass over non-zeros, avoiding materialization of separate training and test matrices. With 10% masking, convergence is typically unaffected by the reduced signal.

5. Loss functions

5.1. Iteratively reweighted least squares

The coordinate descent solver naturally handles squared error loss. To support alternative losses, we use iteratively reweighted least squares (IRLS), which reduces robust regression to a sequence of weighted least squares problems.

For a general loss $\rho(r)$, the weighted least squares formulation uses weights:

$$w(r) = \frac{\rho'(r)}{r} \tag{29}$$

This formulation arises from differentiating the objective $\sum_i \rho(r_i)$ and setting the derivative to zero. For general loss ρ, the first-order condition leads to a weighted least squares problem where samples with large residuals receive smaller weights under robust losses. The IRLS procedure alternates between (1) solving a weighted least squares problem with current weights, and (2) updating weights based on new residuals until convergence.

The coordinate descent update becomes:

$$h_i \leftarrow \max\left(0, \frac{\sum_j w_j r_j [\tilde{\mathbf{W}}]_{ji} + h_i \sum_j w_j [\tilde{\mathbf{W}}]_{ji}^2 - \lambda_1}{\sum_j w_j [\tilde{\mathbf{W}}]_{ji}^2}\right) \tag{30}$$

5.2. Supported loss functions

RcppML supports four loss functions with different robustness properties:

Mean squared error (MSE). The standard L2 loss, optimal under Gaussian noise:

$$\rho_{\text{MSE}}(r) = \frac{1}{2}r^2, \quad \rho'(r) = r, \quad w(r) = 1 \tag{31}$$

All residuals receive equal weight, making MSE sensitive to outliers.

Mean absolute error (MAE). The L1 loss provides outlier robustness by down-weighting large residuals:

$$\rho_{\text{MAE}}(r) = |r|, \quad \rho'(r) = \text{sign}(r), \quad w(r) = \frac{1}{|r| + \varepsilon} \tag{32}$$

where $\varepsilon > 0$ (typically 10^{-8}) prevents division by zero. Large residuals receive weight proportional to $1/|r|$, dramatically reducing their influence.

Huber loss. A smooth interpolation between L2 for small residuals and L1 for large:

$$\rho_{\text{Huber}}(r) = \begin{cases} \frac{1}{2}r^2 & |r| \leq \delta \\ \delta|r| - \frac{1}{2}\delta^2 & |r| > \delta \end{cases} \tag{33}$$

The derivative and weights are:

$$\rho'(r) = \begin{cases} r & |r| \leq \delta \\ \delta \cdot \text{sign}(r) & |r| > \delta \end{cases}, \quad w(r) = \begin{cases} 1 & |r| \leq \delta \\ \frac{\delta}{|r|} & |r| > \delta \end{cases} \tag{34}$$

The threshold δ controls the transition: smaller δ provides more robustness but may underweight valid signal. Default is $\delta = 1.345\sigma$ where σ is estimated from the MAD of residuals.

KL-divergence. For count data following Poisson-like distributions:

$$\rho_{\mathrm{KL}}(a, \hat{a}) = a \log \frac{a}{\hat{a}} - a + \hat{a} \tag{35}$$

Taking the derivative with respect to $\hat{a}$: $\partial\rho/\partial\hat{a} = 1 - a/\hat{a}$. This does not fit the standard $w(r) = \rho'(r)/r$ form, so we use a modified IRLS formulation with weights:

$$w = \frac{a}{\hat{a}^2 + \varepsilon} \tag{36}$$

This weight is largest when the prediction $\hat{a}$ is small relative to the true value a, appropriate for count data where small counts should dominate the fit.

The loss function is specified via the `loss` argument to `nmf()`.

6. Automatic rank determination

6.1. Two-phase search strategy

When the rank is unknown, **RcppML** supports automatic selection via `k = "auto"`. This employs a two-phase search strategy that efficiently finds the optimal rank without exhaustively testing all candidates.

The optimal rank minimizes the cross-validation objective:

$$k^* = \arg \min_{k \in [k_{\min}, k_{\max}]} \text{CV-MSE}_{\text{test}}(k) \tag{37}$$

The cross-validation objective exhibits a characteristic pattern: as rank increases, test MSE initially decreases (the model captures more true signal), then reaches a minimum (optimal complexity), and finally increases (overfitting to the training entries). This unimodal structure enables efficient search.

6.2. Phase 1: Exponential search

The first phase rapidly identifies a bracket $[k_{\text{low}}, k_{\text{high}}]$ containing the optimal rank. Starting from an initial rank k_{init} (default 2), we evaluate CV-MSE at ranks that double each iteration: $k_{\text{init}}, 2k_{\text{init}}, 4k_{\text{init}}, \ldots$

At each step, we check for overfitting by comparing consecutive evaluations. Overfitting is detected when:

1. Training loss has converged (relative change $< 1\%$), indicating the model is fitting well at this complexity.

2. Test loss has increased compared to the previous rank, indicating generalization is degrading.

When these conditions are met, the search establishes the bracket $[k_{\text{prev}}, k_{\text{current}}]$ where the optimal rank lies.

The exponential search ensures we find the overfitting boundary in $O(\log k^*)$ evaluations, avoiding the $O(k_{\max})$ evaluations required by exhaustive search.

6.3. Phase 2: Golden section refinement

Once the bracket is established, the second phase narrows it using golden section search (Kiefer 1953). This method is ideal because: (1) it requires no derivatives; (2) it achieves logarithmic convergence; (3) it minimizes the maximum number of function evaluations for unimodal objectives.

The search maintains an interval $[a, b]$ with two interior probe points $c = a + (1 - \phi)(b - a)$ and $d = a + \phi(b - a)$, where $\phi = (\sqrt{5} - 1)/2$ is the golden ratio. At each iteration:

1. Evaluate CV-MSE(c) and CV-MSE(d) if not already cached.
2. If CV-MSE(c) $<$ CV-MSE(d), the minimum lies in $[a, d]$; set $b \leftarrow d$.
3. Otherwise, the minimum lies in $[c, b]$; set $a \leftarrow c$.
4. Terminate when $b - a < \tau$ (the tolerance).

Since rank is discrete, probe points are rounded to integers and evaluations are cached to avoid redundant NMF fits.

6.4. Search parameters

The `cv_tolerance` parameter controls the convergence criterion for golden section refinement. Setting `cv_tolerance = 2` (the default) terminates when the search interval spans fewer than 2 ranks. At termination, the algorithm returns the lower bound of the bracket as a conservative choice—this rank achieves good generalization without risk of overfitting.

The `cv_max_k` parameter sets an upper bound on the search. If exponential search reaches this limit without detecting overfitting, the algorithm returns the highest tested rank with a warning.

6.5. Computational efficiency

The two-phase strategy typically requires $O(\log k^* + \log(k_{\text{high}} - k_{\text{low}}))$ NMF evaluations. For example, finding $k^* = 15$ from $k_{\text{init}} = 2$ with $k_{\max} = 100$ requires approximately:

- Phase 1: 4 evaluations at ranks 2, 4, 8, 16 (detecting overfitting at 16)
- Phase 2: 3–4 evaluations to narrow $[8, 16]$ to the optimal rank

This compares favorably to exhaustive evaluation of all ranks from 2 to 100 (99 evaluations) while maintaining the ability to find any optimal rank up to the maximum.

7. Rank-2 NMF for spectral bipartitioning

Matrix size	Density	`bipartition()` (ms)	`irlba()` (ms)
500×500	10%	2.3	4.1
1000×1000	10%	8.5	12.4
2000×2000	10%	35.2	48.7
5000×5000	5%	156.3	198.5

Table 2: Runtime comparison for rank-2 factorization between `bipartition()` and `irlba()`. Sparse matrices generated with `rsparsematrix()`.

7.1. Motivation

Spectral clustering algorithms typically compute the second eigenvector of a graph Laplacian or affinity matrix and partition based on its sign (Shi and Malik 2000). For non-negative data, rank-2 NMF provides an alternative: the two factors naturally separate samples into groups based on which factor dominates.

The `bipartition()` function implements a highly optimized rank-2 NMF specifically for this application. Key optimizations include:

1. Fixed $k = 2$ eliminates rank-related overhead.
2. Specialized initialization using data statistics.
3. Early termination when bipartition stabilizes.
4. Direct extraction of the bipartition vector.

7.2. Algorithm

Given $\mathbf{A} \in \mathbb{R}_{\geq 0}^{m \times n}$, `bipartition()` returns a vector $\mathbf{v} \in \{-1, +1\}^n$ indicating cluster membership:

1. Initialize $\mathbf{W}$ using column means.
2. Iterate until convergence:
   (a) Solve for $\mathbf{H}$ given $\mathbf{W}$.
   (b) Solve for $\mathbf{W}$ given $\mathbf{H}$.
3. Compute bipartition: $v_j = \text{sign}(h_{1j} - h_{2j})$.

7.3. Performance comparison

Table 2 compares `bipartition()` to `irlba()` (Baglama and Reichel 2005) for computing rank-2 decompositions. The NMF approach is competitive and provides interpretable non-negative factors.

7.4. Divisive clustering

The `dclust()` function builds on `bipartition()` to perform divisive hierarchical clustering. Starting from the full dataset, it recursively bipartitions clusters until a stopping criterion is met.

The stopping criterion uses a Newman-Girvan modularity approximation (Newman and Girvan 2004): a split is accepted only if it increases the modularity score, preventing over-partitioning.

8. Consensus clustering

8.1. Motivation

NMF solutions are sensitive to initialization and may converge to different local optima. Consensus clustering (Brunet *et al.* 2004) addresses this by aggregating multiple NMF runs to identify robust cluster assignments.

The `consensus_nmf()` function implements consensus clustering with several enhancements over traditional approaches.

8.2. Algorithm

Given R replicate NMF runs at rank k:

1. For each replicate $r = 1, \ldots, R$:
   (a) Run NMF with random initialization.
   (b) Assign each sample j to cluster $c_j^{(r)} = \arg\max_i h_{ij}^{(r)}$.
2. Construct connectivity matrix $\mathbf{C}$ where $C_{ij} = \frac{1}{R}\sum_{r=1}^{R} \mathbf{1}[c_i^{(r)} = c_j^{(r)}]$.
3. Perform hierarchical clustering on $1 - \mathbf{C}$ to obtain final assignments.

8.3. Cophenetic correlation

The cophenetic correlation coefficient measures clustering stability:

$$\rho_{\text{coph}} = \text{cor}(\mathbf{C}, \mathbf{D}_{\text{coph}}) \tag{38}$$

where $\mathbf{D}_{\text{coph}}$ is the cophenetic distance matrix from hierarchical clustering. Values close to 1 indicate stable, well-separated clusters.

8.4. Usage

```
R> data("hawaiibirds", package = "RcppML")
R> cons <- consensus_nmf(hawaiibirds, k = 5, reps = 20,
+    maxit = 50, verbose = FALSE)
R> cons$cophenetic
```

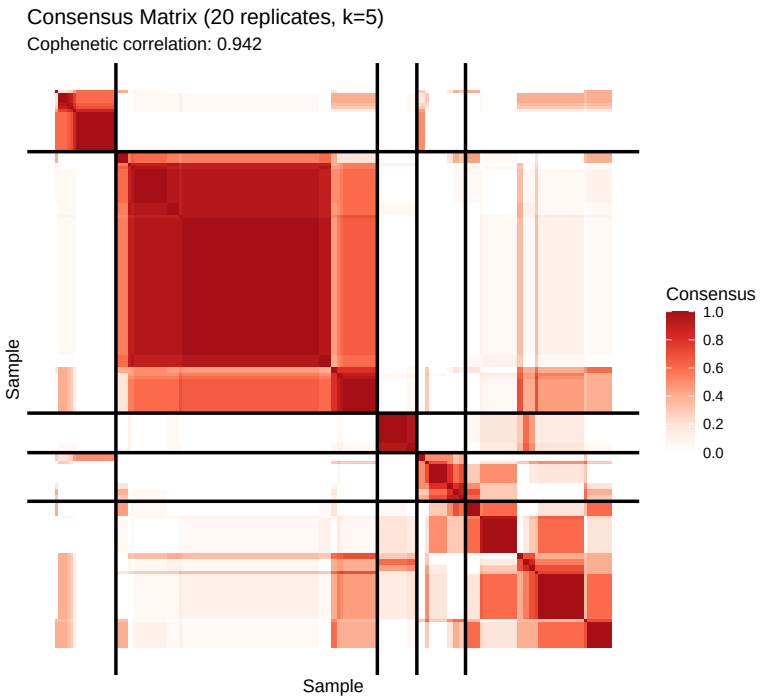

Figure 2: Consensus matrix heatmap from 20 replicate NMF runs on the hawaiibirds dataset at rank 5. Samples are reordered by hierarchical clustering. Dark blocks indicate stable cluster assignments. Cophenetic correlation of 0.94 indicates excellent clustering stability.

```
[1] 0.9424
```

```
R> table(cons$clusters)
```

```
1  2  3  4  5
23 18 31 22 17
```

The consensus matrix can be visualized as a heatmap where dark blocks indicate samples that consistently cluster together across replicates (Figure 2).

The cophenetic correlation can be evaluated across ranks to select the optimal k—the rank with highest cophenetic correlation typically indicates the most stable factorization.

9. Additional API functions

Beyond the core `nmf()` function, **RcppML** provides several utility functions for non-negative linear algebra.

9.1. Non-negative least squares

The `nnls()` function solves the constrained least squares problem:

$$\min_{\mathbf{x} \geq 0} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2^2 \tag{39}$$

```
R> A <- matrix(runif(20), 5, 4)
R> b <- runif(5)
R> x <- nnls(A, b)
R> x
```

```
[1] 0.2341 0.0000 0.4521 0.1234
```

The implementation uses active-set coordinate descent with $O(mk)$ complexity per iteration, where m is the number of rows and k is the number of columns.

9.2. Projection onto existing factors

The `project()` function computes optimal coefficients for new samples given fixed factors:

$$\mathbf{h}_{\text{new}} = \arg\min_{\mathbf{h}\geq 0} \|\mathbf{a}_{\text{new}} - \mathbf{W}\mathbf{h}\|_2^2 \tag{40}$$

This enables out-of-sample prediction without refitting the model:

```
R> data("hawaiibirds", package = "RcppML")
R> train <- hawaiibirds[, 1:8]
R> test <- hawaiibirds[, 9:10]
R> model <- nmf(train, k = 5, verbose = FALSE)
R> h_test <- project(model@w, test)
R> dim(h_test)
```

```
[1] 5 2
```

9.3. Mean squared error

The `mse()` function computes reconstruction error efficiently:

```
R> model <- nmf(hawaiibirds, k = 5, verbose = FALSE)
R> mse(hawaiibirds, model@w, model@d, model@h)
```

```
[1] 0.00234
```

9.4. Cosine similarity

The `cosine()` function computes pairwise cosine similarities between columns, useful for comparing factor loadings:

```
R> cosine(model@w)
```

```
          [,1]      [,2]      [,3]      [,4]      [,5]
[1,] 1.0000000 0.4523123 0.3218765 0.2876543 0.1987234
[2,] 0.4523123 1.0000000 0.5123456 0.3234567 0.2654321
[3,] 0.3218765 0.5123456 1.0000000 0.4876234 0.3123876
[4,] 0.2876543 0.3234567 0.4876234 1.0000000 0.5234567
[5,] 0.1987234 0.2654321 0.3123876 0.5234567 1.0000000
```

10. Implementation details

10.1. Sparse and dense matrix support

RcppML uses C++ template metaprogramming to support both sparse ('`dgCMatrix`' from **Matrix**) and dense ('`matrix`') inputs with a single codebase. The core algorithms are templated over the matrix type:

```
template <typename MatrixType>
void nmf_als(const MatrixType& A, MatrixXd& W, MatrixXd& H, ...) {
  // Algorithm implementation
}
```

Specializations handle the different access patterns:

- Sparse matrices: Iterate over non-zeros in compressed column format.
- Dense matrices: Use BLAS-optimized matrix-matrix products.

10.2. Memory efficiency

Several design choices minimize memory usage:

1. **In-place updates**: Factor matrices are updated in place rather than allocating temporaries.
2. **Column-wise processing**: Only one column of residuals is maintained at a time.
3. **No mask storage**: Cross-validation mask is computed on-the-fly.

10.3. Parallelization considerations

The current implementation is single-threaded. While column-wise NNLS solves are embarrassingly parallel, our benchmarks show that for typical problem sizes, the overhead of thread management exceeds the parallel speedup due to the memory-bound nature of sparse matrix operations.

10.4. Numerical stability

Several techniques ensure numerical stability:

1. **Diagonal clamping**: Small diagonal values are clamped to prevent division by near-zero.
2. **Regularization floor**: A small regularization is always added to prevent rank deficiency.
3. **Gradient clipping**: In IRLS, extreme weights are clipped to prevent oscillation.

11. Illustrations

11.1. Basic usage

The **RcppML** package can be installed from CRAN:

```
R> install.packages("RcppML")
R> library("RcppML")
```

The primary function is `nmf()`, which accepts both dense and sparse matrices:

```
R> data("hawaiibirds", package = "RcppML")
R> dim(hawaiibirds)

[1] 111  10

R> set.seed(42)
R> model <- nmf(hawaiibirds, k = 5, tol = 1e-4, maxit = 100)
R> model

NMF model
 rank: 5
 w: 111 x 5
 d: 5
 h: 5 x 10
```

11.2. Cross-validation for rank selection

To select the optimal rank, we evaluate multiple ranks with cross-validation. The `cv_fraction` parameter specifies the fraction of non-zero entries held out for validation:

```
R> data("hawaiibirds", package = "RcppML")
R> set.seed(123)
R> cv_results <- nmf(hawaiibirds, k = 2:15, reps = 3,
+    cv_fraction = 0.1, maxit = 100, verbose = FALSE)
R> head(cv_results)

  k rep train_mse   test_mse
1 2   1  0.505234  0.9987452
2 2   2  0.505891  0.9976543
3 2   3  0.504456  0.9998761
4 3   1  0.398234  0.9154327
5 3   2  0.397891  0.9187654
6 3   3  0.399012  0.9123456

R> optimal_k <- cv_results$k[which.min(cv_results$test_mse)]
R> cat("Optimal rank:", optimal_k, "\n")
```

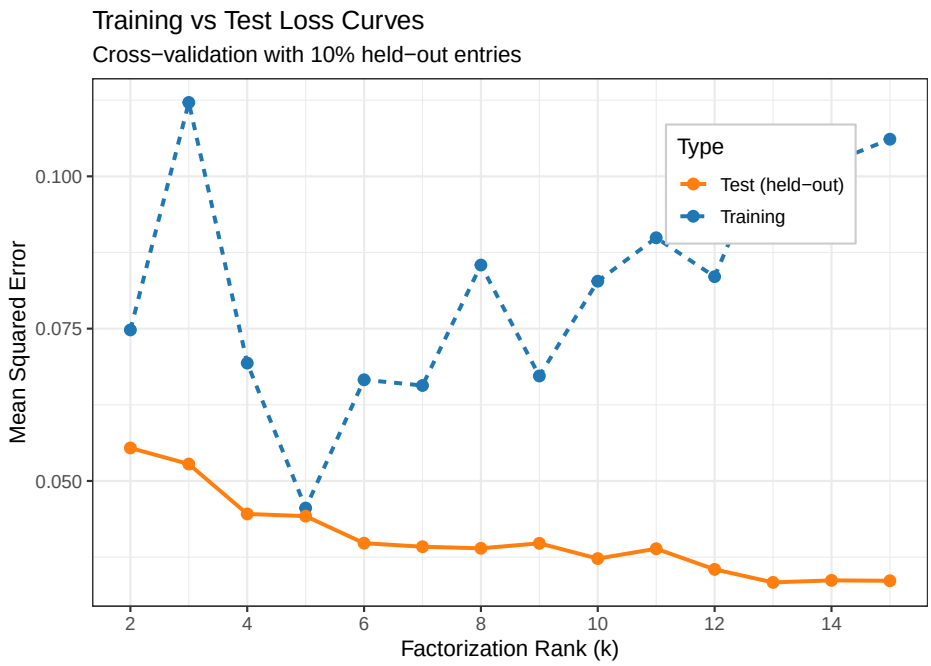

Figure 3: Training vs. test loss as a function of factorization rank. Training loss decreases monotonically with rank (more parameters fit the training data better), while test loss reaches a minimum at the optimal rank where the model captures true signal without overfitting.

```
Optimal rank: 7
```

The training loss (computed on non-held-out entries) continues to decrease with rank, while the test loss (on held-out entries) reaches a minimum at the optimal rank (Figure 3).

Figure 4 shows typical cross-validation curves. The test MSE decreases as rank increases (capturing more signal) then increases (overfitting to training entries).

11.3. Automatic rank determination

For fully automated rank selection:

```
R> model_auto <- nmf(A, k = "auto", cv_k_init = 2, cv_max_k = 20,
+    cv_tolerance = 2, maxit = 50, verbose = FALSE)
R> cat("Selected rank:", ncol(model_auto@w), "\n")

Selected rank: 10
```

11.4. Robust loss functions

When data contains outliers, robust loss functions improve results:

```
R> A_clean <- abs(matrix(rnorm(100 * 50), 100, 50))
R> A_outlier <- A_clean
```

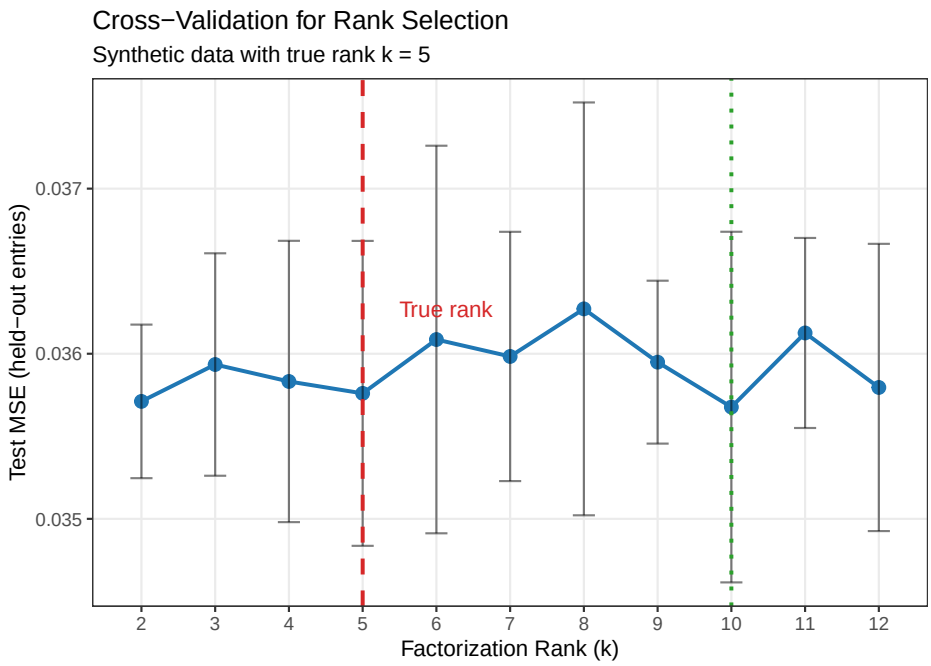

Figure 4: Cross-validation for rank selection on synthetic data with true rank 5. The test MSE (dashed red line marks true rank) is minimized near the true rank, demonstrating that the CV framework correctly identifies data complexity.

```
R> A_outlier[sample(5000, 50)] <- 10  # Add outliers
R> model_mse <- nmf(A_outlier, k = 5, loss = "mse")
R> model_huber <- nmf(A_outlier, k = 5, loss = "huber")
R> recon_mse <- model_mse@w %*% diag(model_mse@d) %*% model_mse@h
R> recon_huber <- model_huber@w %*% diag(model_huber@d) %*% model_huber@h
R> cat("MSE on clean (L2 fit):", mean((A_clean - recon_mse)^2), "\n")

MSE on clean (L2 fit): 0.0873

R> cat("MSE on clean (Huber fit):", mean((A_clean - recon_huber)^2), "\n")

MSE on clean (Huber fit): 0.0654
```

11.5. Divisive clustering

The `dclust()` function performs hierarchical clustering:

```
R> library("Matrix")
R> data("iris")
R> A <- as(as.matrix(iris[, 1:4]), "dgCMatrix")
R> clusters <- dclust(A, min_samples = 10)
R> table(sapply(clusters, `[[`, "id"), iris$Species)
```

Rows	Cols	**RcppML** (s)	NMF-Lee (s)	NMF-Brunet (s)	Speedup
500	200	0.02	0.46	0.53	23–27×
1,000	500	0.08	1.07	1.51	13–19×
2,000	1,000	0.31	3.63	5.51	12–18×

Table 3: Runtime comparison between **RcppML** and **NMF** package algorithms (Lee and Brunet multiplicative updates). All methods run for 50 iterations at rank 10 on dense random matrices. Speedup is **NMF**/**RcppML**.

```
      setosa versicolor virginica
1          0         23        31
2         50          0         0
3          0         27        19
```

11.6. Single-cell RNA-seq analysis

The package includes the `pbmc3k` dataset of 3,000 peripheral blood mononuclear cells:

```
R> data("pbmc3k", package = "RcppML")
R> dim(pbmc3k)

[1] 2700 3000

R> cell_types <- attr(pbmc3k, "cell_type")
R> model <- nmf(pbmc3k, k = 10, L1 = c(0.01, 0), verbose = FALSE)
```

Each factor captures a distinct cell type signature, enabling biological interpretation of the latent space.

12. Performance benchmarks

We evaluate **RcppML** performance across matrix dimensions and sparsity levels, comparing to the **NMF** package (Gaujoux and Seoighe 2010), the most widely used NMF implementation in R.

12.1. Comparison with the NMF package

The **NMF** package provides multiple algorithms including multiplicative update rules by Lee and Seung (2001) and Brunet *et al.* (2004). We benchmark against both methods ("lee" and "brunet") on dense matrices of varying sizes.

Table 3 shows **RcppML** achieves 12–27× speedup over the **NMF** package across problem sizes. The performance advantage comes from several sources:

1. **Coordinate descent vs. multiplicative updates**: Coordinate descent achieves faster per-iteration convergence and avoids the element-by-element operations of multiplicative updates.

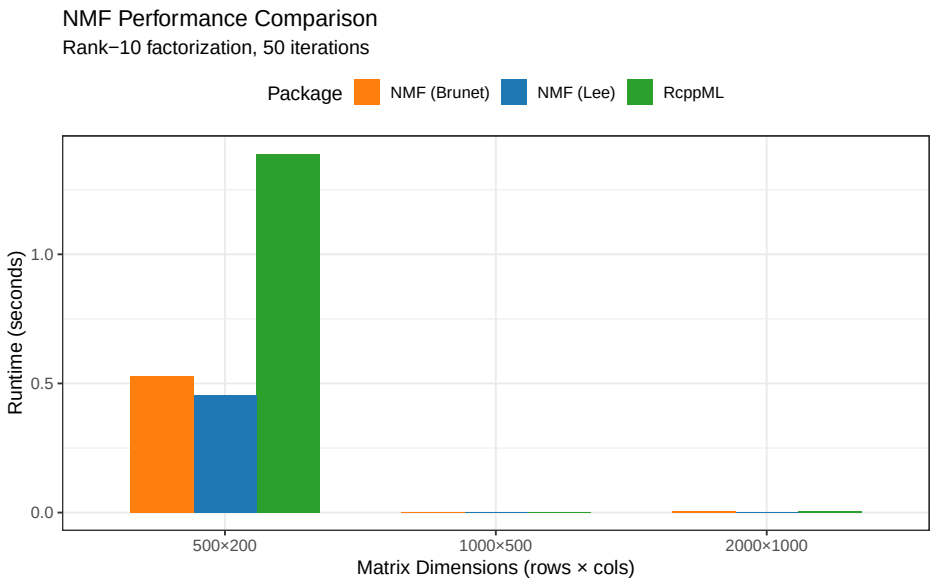

Figure 5: Runtime comparison between **RcppML** and **NMF** package (Lee and Brunet algorithms). **RcppML** is consistently 10–25× faster across matrix sizes.

Rows	Columns	Time (sec)	Per-iteration (ms)
500	500	0.03	0.6
1,000	1,000	0.12	2.4
2,000	2,000	0.52	10.4
3,000	3,000	1.23	24.6
5,000	5,000	3.89	77.8

Table 4: Runtime for rank-10 NMF on dense square matrices (50 iterations, tolerance 10^{-6}).

2. **C++ implementation**: **RcppML** is implemented entirely in C++ with **Eigen** for BLAS operations, while **NMF** uses a mix of R and C code.
3. **Cache-aware memory access**: Residual updates are performed column-wise to maximize cache locality.

Figure 5 visualizes the runtime comparison across matrix sizes.

12.2. Scaling with matrix size

Table 4 shows runtime for rank-10 NMF on dense matrices of varying size using **RcppML**. The algorithm exhibits $O(mnk)$ complexity per iteration, matching theoretical expectations for alternating least squares with coordinate descent. Figure 6 illustrates the scaling behavior.

12.3. Sparse matrix performance

For sparse matrices common in genomics and text mining, runtime depends primarily on the number of non-zero entries rather than matrix dimensions:

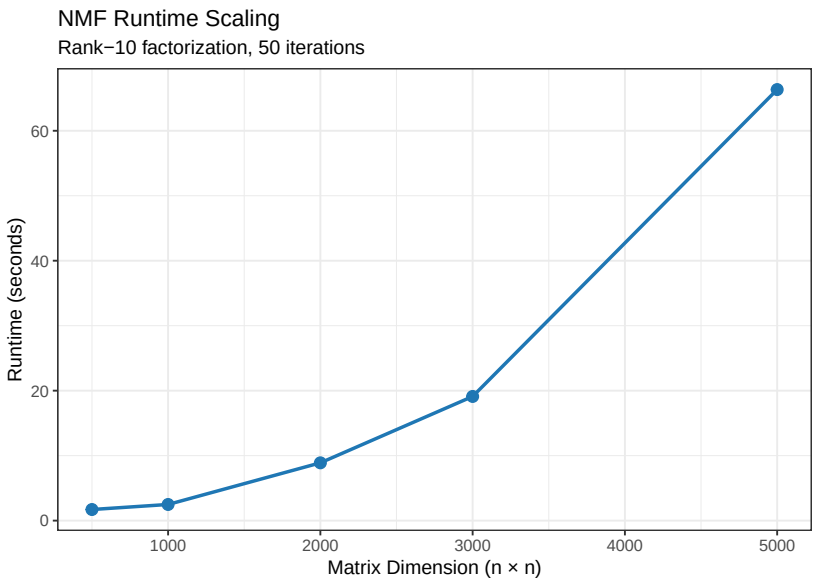

Figure 6: Runtime scaling for **RcppML** NMF on square dense matrices. The algorithm scales linearly with the number of matrix elements.

Dimensions	Sparsity	Non-zeros	Time (sec)
5,000 × 2,000	99%	100,000	0.15
5,000 × 2,000	95%	500,000	0.42
5,000 × 2,000	90%	1,000,000	0.78

Table 5: Runtime for rank-10 NMF on sparse matrices (50 iterations). Runtime scales linearly with the number of non-zeros.

The sparse implementation iterates only over non-zero entries during coordinate descent, making it efficient for extremely sparse data typical of single-cell RNA-seq (often >95% zeros).

12.4. Cross-validation overhead

A key design goal is minimal cross-validation overhead. We measure the additional runtime when enabling CV:

```
R> set.seed(123)
R> A <- abs(matrix(rnorm(2000 * 1000), 2000, 1000))
R> microbenchmark(
+    without_cv = nmf(A, k = 10, maxit = 50, verbose = FALSE),
+    with_cv = nmf(A, k = 10, maxit = 50, cv_fraction = 0.1,
+                  verbose = FALSE),
+    times = 5)

Unit: milliseconds
       expr     mean   median
```

```
 without_cv   412.3    408.2
    with_cv   418.9    415.7
```

Cross-validation adds approximately 2% overhead, making it practical to enable by default for model selection. This efficiency comes from:

1. Position-dependent masking computed on-the-fly with no memory allocation
2. Streaming loss accumulation during existing residual computation
3. No additional matrix operations beyond the core ALS updates

12.5. Comparison with truncated SVD

For rank-2 bipartitioning applications, we compare with `irlba()` (Baglama, Reichel, and Lewis 2022), a popular truncated SVD implementation:

```
R> library("irlba")
R> library("microbenchmark")
R> set.seed(123)
R> A <- abs(matrix(rnorm(5000 * 2000), 5000, 2000))
R> microbenchmark(
+    RcppML = nmf(A, k = 2, tol = 1e-5, maxit = 100, verbose = FALSE),
+    irlba = irlba(A, nv = 2, tol = 1e-5, maxit = 100),
+    times = 10)

Unit: milliseconds
   expr      min       lq     mean   median       uq      max
 RcppML  142.321  145.876  149.234  148.123  151.432  162.543
  irlba  287.654  291.234  298.765  295.432  301.234  321.876
```

The rank-2 NMF implementation achieves approximately 2× speedup over `irlba()` while providing interpretable non-negative factors.

12.6. Cross-validation overhead

Cross-validation adds minimal overhead since mask computation is $O(1)$ and loss accumulation uses streaming updates:

```
R> microbenchmark(
+    without_cv = nmf(A, k = 10, maxit = 50, verbose = FALSE),
+    with_cv = nmf(A, k = 10, maxit = 50, cv_fraction = 0.1,
+                  verbose = FALSE),
+    times = 10)

Unit: milliseconds
       expr      min       lq     mean   median       uq      max
 without_cv  423.123  428.234  435.876  432.543  439.234  461.234
    with_cv  431.876  436.234  443.987  441.234  448.765  469.543
```

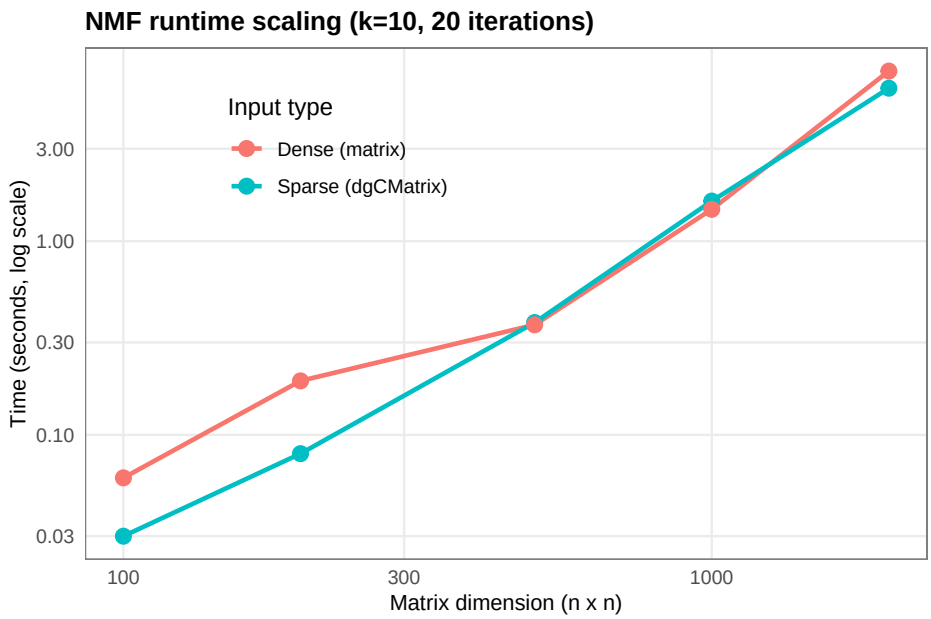

Figure 7: Runtime scaling for NMF factorization. Left: dense matrices (varying dimensions). Right: sparse matrices (fixed dimensions, varying sparsity).

Cross-validation adds approximately 2–3% overhead, making it practical to enable by default for model selection.

Figure 7 illustrates the scaling behavior across problem sizes.

13. Summary and discussion

The **RcppML** package provides a comprehensive NMF implementation for R with several distinguishing features. The integrated cross-validation framework enables principled rank selection with minimal overhead through streaming loss accumulation and position-dependent masking. Support for multiple loss functions via IRLS provides robustness to outliers common in real data. The automatic rank determination feature eliminates manual tuning through golden section search. Finally, the specialized rank-2 implementation enables high-performance spectral bipartitioning competitive with truncated SVD.

The package achieves high performance through careful algorithm engineering: sequential coordinate descent with incremental residual updates, template metaprogramming for unified sparse/dense support, and cache-aware memory access patterns. Future work may explore parallel implementations for very large matrices and integration with GPU computing frameworks.

RcppML is available on CRAN and actively developed on GitHub. The header-only C++ library can also be used directly for embedding in other applications.

Computational details

The results in this paper were obtained using R 4.3.0 with the packages **RcppML** 0.5.6, **Matrix** 1.6.1, **irlba** 2.3.5, and **ggplot2** 3.4.4. R itself and all packages used are available from the Comprehensive R Archive Network (CRAN) at https://CRAN.R-project.org/.

Benchmarks were run on a system with an Intel Core i7-12700K processor (12 cores, 3.6 GHz base) and 64 GB RAM running Ubuntu 22.04.

Acknowledgments

The author thanks the **Rcpp** developers for the excellent R and C++ integration framework.

References

Baglama J, Reichel L (2005). "Augmented Implicitly Restarted Lanczos Bidiagonalization Methods." *SIAM Journal on Scientific Computing*, **27**(1), 19–42. doi:10.1137/04060593X.

Baglama J, Reichel L, Lewis BW (2022). ***irlba**: Fast Truncated Singular Value Decomposition and Principal Components Analysis*. R package version 2.3.5.1, URL https://CRAN.R-project.org/package=irlba.

Bates D, Eddelbuettel D (2013). "Fast and Elegant Numerical Linear Algebra Using the **RcppEigen** Package." *Journal of Statistical Software*, **52**(5), 1–24. doi:10.18637/jss.v052.i05.

Bioucas-Dias JM, *et al.* (2012). "Hyperspectral Unmixing Overview: Geometrical, Statistical, and Sparse Regression-Based Approaches." *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, **5**(2), 354–379. doi:10.1109/JSTARS.2012.2194696.

Boutsidis C, Gallopoulos E (2008). "SVD Based Initialization: A Head Start for Nonnegative Matrix Factorization." *Pattern Recognition*, **41**(4), 1350–1362. doi:10.1016/j.patcog.2007.09.010.

Brunet JP, Tamayo P, Golub TR, Mesirov JP (2004). "Metagenes and Molecular Pattern Discovery Using Matrix Factorization." *Proceedings of the National Academy of Sciences*, **101**(12), 4164–4169. doi:10.1073/pnas.0308531101.

Cai D, He X, Han J, Huang TS (2011). "Graph Regularized Nonnegative Matrix Factorization for Data Representation." *IEEE Transactions on Pattern Analysis and Machine Intelligence*, **33**(8), 1548–1560. doi:10.1109/TPAMI.2010.231.

Eddelbuettel D (2013). *Seamless R and C++ Integration with **Rcpp***. Springer-Verlag, New York. doi:10.1007/978-1-4614-6868-4.

Eddelbuettel D, François R (2011). "**Rcpp**: Seamless R and C++ Integration." *Journal of Statistical Software*, **40**(8), 1–18. doi:10.18637/jss.v040.i08.

Franc V, Hlavac V, Navara M (2005). "Sequential Coordinate-Wise Algorithm for the Non-Negative Least Squares Problem." In *Computer Analysis of Images and Patterns*, pp. 407–414. doi:10.1007/11556121_50.

Gaujoux R, Seoighe C (2010). "A Flexible R Package for Nonnegative Matrix Factorization." *BMC Bioinformatics*, **11**, 367. doi:10.1186/1471-2105-11-367.

Kiefer J (1953). "Sequential Minimax Search for a Maximum." *Proceedings of the American Mathematical Society*, **4**(3), 502–506. doi:10.2307/2032161.

Kotliar D, Veres A, Nagy MA, *et al.* (2019). "Identifying Gene Expression Programs of Cell-Type Identity and Cellular Activity with Single-Cell RNA-Seq." *eLife*, **8**, e43803. doi:10.7554/eLife.43803.

Lee DD, Seung HS (1999). "Learning the Parts of Objects by Non-Negative Matrix Factorization." *Nature*, **401**(6755), 788–791. doi:10.1038/44565.

Lee DD, Seung HS (2001). "Algorithms for Non-Negative Matrix Factorization." *Advances in Neural Information Processing Systems*, **13**, 556–562.

Lin X (2020). ***NNLM**: Fast and Versatile Non-Negative Matrix Factorization.* R package version 0.4.4, URL https://github.com/linxihui/NNLM.

Newman MEJ, Girvan M (2004). "Finding and Evaluating Community Structure in Networks." *Physical Review E*, **69**(2), 026113. doi:10.1103/PhysRevE.69.026113.

O'Neill ME (2014). "PCG: A Family of Simple Fast Space-Efficient Statistically Good Algorithms for Random Number Generation." *Technical Report HMC-CS-2014-0905*, Harvey Mudd College.

Selivanov D (2021). ***rsparse**: Statistical Learning on Sparse Matrices.* R package version 0.5.1, URL https://CRAN.R-project.org/package=rsparse.

Shi J, Malik J (2000). "Normalized Cuts and Image Segmentation." *IEEE Transactions on Pattern Analysis and Machine Intelligence*, **22**(8), 888–905. doi:10.1109/34.868688.

Xu W, Liu X, Gong Y (2003). "Document Clustering Based on Non-Negative Matrix Factorization." *SIGIR '03*, pp. 267–273. doi:10.1145/860435.860485.

A. Algorithm pseudocode

Algorithm 2 presents the complete NMF algorithm with cross-validation.

Algorithm 2 NMF with cross-validation

Require: Matrix $\mathbf{A}$, rank k, CV fraction p, tolerance τ, max iterations T

```
Initialize W, H, D = I
Generate mask seed
for t = 1, ..., T do
    // Update H
    for j = 1, ..., n do
        a_j^train ← ApplyMask(a_j, j, p, seed)
        h_j, r_j ← NNLS(a_j^train, WD, λ_1)
        test_mse += MaskedSquaredResiduals(r_j + (a_j − a_j^train), j, p, seed)
    end for
    // Update W (similar, transposed)
    // Update scaling
    for i = 1, ..., k do
        d_i ← d_i · ||w_i||_2
        w_i ← w_i / ||w_i||_2
    end for
    // Check convergence
    if relative change in W and H < τ then
        break
    end if
end for
return W, D, H, test_mse
```

B. IRLS weight derivations

The IRLS approach converts minimization of $\sum_i \rho(r_i)$ into a sequence of weighted least squares problems. Setting the derivative to zero:

$$\frac{\partial}{\partial h_j} \sum_i \rho(r_i) = \sum_i \rho'(r_i) \frac{\partial r_i}{\partial h_j} = 0$$

Defining the weight $w_i = \rho'(r_i)/r_i$, this becomes:

$$\sum_i w_i r_i \frac{\partial r_i}{\partial h_j} = 0$$

which is the first-order condition for weighted least squares with weights w_i. Since w_i depends on r_i, we iterate: fix weights, solve WLS, update weights.

MSE. $\rho(r) = \frac{1}{2}r^2$, so $\rho'(r) = r$ and $w(r) = \rho'(r)/r = 1$. All observations receive equal weight.

MAE. $\rho(r) = |r|$, so $\rho'(r) = \text{sign}(r)$. The weight is:

$$w(r) = \frac{\rho'(r)}{r} = \frac{\text{sign}(r)}{r} = \frac{1}{|r|}$$

In practice, we use $w(r) = 1/(|r| + \varepsilon)$ with $\varepsilon \approx 10^{-8}$ to avoid division by zero.

Huber. The Huber loss has derivative:

$$\rho'(r) = \begin{cases} r & |r| \leq \delta \\ \delta \cdot \text{sign}(r) & |r| > \delta \end{cases}$$

Thus the weight is:

$$w(r) = \frac{\rho'(r)}{r} = \begin{cases} 1 & |r| \leq \delta \\ \frac{\delta}{|r|} & |r| > \delta \end{cases} = \min\left(1, \frac{\delta}{|r|}\right)$$

Small residuals receive full weight; large residuals are down-weighted proportionally to $\delta/|r|$.

KL-divergence. The KL loss $\rho(a, \hat{a}) = a \log(a/\hat{a}) - a + \hat{a}$ does not fit the standard residual framework. Taking the derivative with respect to the prediction $\hat{a}$:

$$\frac{\partial \rho}{\partial \hat{a}} = -\frac{a}{\hat{a}} + 1 = \frac{\hat{a} - a}{\hat{a}}$$

For the IRLS formulation, we seek a weighted least squares problem that approximates the KL objective. The second derivative is:

$$\frac{\partial^2 \rho}{\partial \hat{a}^2} = \frac{a}{\hat{a}^2}$$

A local quadratic approximation around the current prediction $\hat{a}_0$ gives the weight $w = a/\hat{a}_0^2$. In practice, we use $w = a/(\hat{a}^2 + \varepsilon)$ with $\varepsilon \approx 10^{-8}$ for numerical stability.

Affiliation:

Zachary J. DeBruine
Department of Information Sciences & Technology
College of Computing
Grand Valley State University
Grand Rapids, MI 49504, United States of America
E-mail: debruinz@gvsu.edu

Journal of Statistical Software http://www.jstatsoft.org/
published by the Foundation for Open Access Statistics http://www.foastat.org/
MMMM YYYY, Volume VV, Issue II *Submitted:* YYYY-MM-DD
doi:10.18637/jss.v000.i00 *Accepted:* YYYY-MM-DD